

Présentation Technique PHP

Professionalisation & Outils Modernes

1. Fonctions Anonymes & Closures

2. Fonctions Natives array_*

3. Composer

4. Packagist.org

01

Fonctions Anonymes, Fléchées & Closures

Une fonction anonyme est une fonction sans nom, assignable à une variable ou passée comme argument.

Anonyme

```
$greet = function($name) {  
    return 'Bonjour ' . $name;  
};  
echo $greet('Assane');
```

Fléchée (Arrow fn)

```
$double = fn($x) => $x * 2;  
echo $double(5); // 10
```

Closure

```
$msg = 'Hello';  
$fn = function() use ($msg) {  
    echo $msg;  
};  
$fn();
```

01

Cas d'usage concrets

Callback dans array_map

```
$res = array_map(fn($x) => $x * 2, [1, 2, 3]);  
// [2, 4, 6]
```

Filtre avec use (Closure)

```
$min = 500;  
$res = array_filter($wallets, fn($w) => $w["solde"] >= $min);
```

Tri avec usort

```
usort($wallets, fn($a, $b) => $a["solde"] <=> $b["solde"]);
```

Différence : fn() vs function()

```
// fn() capture auto les variables parentes  
// function() nécessite use ($var)
```

02

Fonctions Natives PHP sur les Tableaux

`array_map()`

Applique une fonction sur chaque élément

```
array_map(fn($x) => $x * 2, $arr)
```

`array_filter()`

Garde les éléments qui passent un test

```
array_filter($arr, fn($x) => $x > 100)
```

`array_search()`

Cherche une valeur et retourne sa clé

```
array_search('77001122', $phones)
```

`in_array()`

Vérifie si une valeur existe dans un tableau

```
in_array('77001122', $phones)
```

`array_values()`

Réindexe les clés après un filter

```
array_values(array_filter($arr, ...))
```

02

Avant (boucle) vs Après (natif)

✗ Partie A — Boucle manuelle

```
function trouverWallet($wallets, $tel) {  
    $result = null;  
    foreach ($wallets as $w) {  
        if ($w['tel'] === $tel) {  
            $result = $w;  
            break;  
        }  
    }  
    return $result;  
}
```



✓ Partie B — Fonction native

```
function trouverWallet($wallets, $tel) {  
    $results = array_filter(  
        $wallets,  
        fn($w) => $w['tel'] === $tel  
    );  
    return array_values($results)[0] ?? null;  
}
```

03

Composer — Gestionnaire de Dépendances PHP

Composer est l'outil standard de gestion des dépendances en PHP. Il permet d'installer, mettre à jour et autocharger des bibliothèques tierces via un fichier composer.json.

Installation

```
composer require vendor/package
```

Installe le paquet dans /vendor

Mise à jour

```
composer update
```

Met à jour toutes les dépendances

Autoload

```
require 'vendor/autoload.php';
```

Charge automatiquement toutes les classes

composer.json

```
"require": {  
  "php": ">=8.0"  
}
```

04

Packagist.org — Le Registre PHP

Packagist.org est le dépôt central de paquets PHP utilisé par Composer. Il recense plus de 400 000 paquets open source téléchargeables librement.



Paquets populaires sur Packagist



monolog/monolog

Logging PHP



guzzlehttp/guzzle

Client HTTP



symfony/console

CLI applications



phpunit/phpunit

Tests unitaires

Conclusion

01 Les fonctions anonymes & closures permettent un code plus flexible et expressif

02 Les fonctions `array_*` remplacent avantageusement les boucles manuelles (code plus court et lisible)

03 Composer automatise la gestion des dépendances et l'autoloading des classes

04 Packagist.org est l'écosystème central PHP avec plus de 400 000 paquets disponibles